23. Linked Structures, Stacks, and Queues

23. Linked Structures, Stacks, and Queues

Phase: Phase 4 — Algorithms and data structures
Estimated time: 1–2 days
Slug: `23-linked-stacks-queues`

Learning objectives

- Implement stack and queue with slices or lists
- Know when linked lists help vs hurt
- Reason about amortized append costs
- Use containers/list only when needed

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Stack (LIFO)

Slice-backed stacks are idiomatic and cache-friendly:

```
type Stack []int
func (s *Stack) Push(v int) { *s = append(*s, v) }
func (s *Stack) Pop() (int, bool) {
    if len(*s) == 0 { return 0, false }
    v := (*s)[len(*s)-1]
    *s = (*s)[:len(*s)-1]
    return v, true
}
```

Queue (FIFO)

Can use slice with head index, ring buffer, or two stacks. Avoid O(n) `dequeue` from front without strategy.

Linked lists

Useful for O(1) splice given a node; often slower than slices due to pointer chasing. Prefer slices first.

Common mistakes (especially from Python instincts)

- Always reaching for linked lists from textbook habit
- Queue via slice[1:] in hot path causing GC pressure
- Forgetting empty checks on Pop

Exercises

Work in `exercises/23-linked-stacks-queues/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/23-linked-stacks-queues/` until you have an honest failing attempt.

Self-check

1. LIFO vs FIFO?
2. Why are slices often faster than lists?
3. Amortized cost of append?

Further reading

- https://pkg.go.dev/container/list